

รายงานความคืบหน้าโครงการ Disney Lorcana PlayLab Cloud (Sprint 1–3)

วิชา: Cloud Technology (KMITL)

สถานะปัจจุบัน: เสร็จสิ้น Sprint 1–3 (พร้อมสำหรับส่งรายงานความก้าวหน้า Stage 2 วันที่ 10 ก.ย.)

วันที่อัปเดต: 14 สิงหาคม 2026

1. สรุปความคืบหน้า Sprint 1–3

การพัฒนาระบบ Disney Lorcana PlayLab Cloud ดำเนินการมาถึง Sprint 3 แล้ว โดยมุ่งเน้นที่การสร้าง Core Gameplay UI, ระบบ Authentication & Deck Management และระบบ Real-time Multiplayer ด้วย WebSocket ซึ่งสามารถนำไป Deploy และทดสอบการทำงานบน AWS Cloud ได้สำเร็จในเบื้องต้น

1.1. ผลการดำเนินงานแต่ละ Sprint

- **Sprint 1 (Board UI):** พัฒนา Frontend อย่างสมบูรณ์ (React/TypeScript) มีการสร้างไฟล์ `LorcanaBoard.tsx` รองรับระบบ Drag-and-Drop พร้อมชุดข้อมูลการ์ดตั้งต้นกว่า 3,242 ใบ (`cardPool.ts`) โค้ดผ่านการทำ Type Checking (`tsc`) และ Build สำเร็จ
- **Sprint 2 (Auth+Deck):** พัฒนา Backend Microservices สำหรับจัดการผู้ใช้และการ์ด (Node.js/AWS Lambda) มีระบบ Login/Register แบบเข้ารหัส Password ด้วย `bcrypt` และออก Token ด้วย `JWT` รวมทั้งระบบสร้าง/ดึง/ลบการ์ดสำเร็จ ได้รับการ Deploy ขึ้น AWS Learner Lab ผ่าน HTTP API Gateway เรียบร้อยและใช้งานได้จริง
- **Sprint 3 (WebSocket):** พัฒนาระบบห้องเล่นการ์ดแบบ Real-time โดยใช้ AWS API Gateway WebSocket API และ AWS Lambda (`backend/room/handler.ts`) ร่วมกับ Amazon DynamoDB (`RoomStateTable`) ในการจัดการ Connection และสถานะของห้อง สามารถเชื่อมต่อและรับส่งข้อมูล (เช่น การเข้าห้อง, การขยับการ์ด) ระหว่างผู้เล่นได้สำเร็จ

1.2. ข้อจำกัดและการแก้ปัญหา (AWS Learner Lab)

เนื่องจากข้อจำกัดของสภาพแวดล้อม AWS Learner Lab ที่ไม่สามารถใช้สิทธิ์ `iam:CreateRole` ได้ การใช้งาน AWS SAM จึงถูกจำกัด เราได้แก้ปัญหาโดยใช้ `LabRole` ที่มีให้ และปรับใช้สคริปต์ `deploy_manual.sh` ในการอัปเดต Lambda Function ด้วยตัวเองแทน ทำให้สามารถ Deploy ระบบได้อย่างต่อเนื่องตาม Best Practice แบบยืดหยุ่น

2. ตารางสถานะ Sprint เทียบกำหนดการส่งงาน

ระยะเวลา/กำหนดการ	กิจกรรม / งานที่ต้องส่ง	สถานะ
Sprint 1–3 (ถึงปัจจุบัน)	Board UI, Auth/Deck Lambda, WebSocket API	เสร็จสิ้น (100%)
Sprint 4–5 (ถึง 10 ก.ย.)	Deck Analyzer, Integration, เตรียมรายงาน	กำลังดำเนินการ
10 ก.ย. 2026	ส่งความก้าวหน้า Stage 2 (12 คะแนน)	ใกล้ถึงกำหนด
22–26 ก.ย. 2026	นำเสนอ Stage 2 (3 คะแนน)	-
Sprint 6–7 (ถึง 10 ต.ค.)	WebSocket Auto-reconnect, ทดลองโหลด, สรุปผล	-
10 ต.ค. 2026	ส่งผลการทดลอง Stage 3 (10 คะแนน)	-
20–25 ต.ค. 2026	นำเสนอ Stage 3 (5 คะแนน)	-

3. โครงสร้างระบบ Cloud AWS (Architecture)

ปัจจุบันระบบทำงานบนสถาปัตยกรรม Serverless (Microservices) ดังนี้:

- **Compute:** AWS Lambda ทั้งสิ้น 4 ฟังก์ชันหลัก (Login, Register, Deck Management, Room Handler)
- **API Routing:**
 - HTTP API Gateway สำหรับ RESTful endpoints (Auth & Deck) มีทั้งหมด 5 Routes
 - WebSocket API Gateway (`wss://a86238wqo4.execute-api.us-east-1.amazonaws.com/prod`) สำหรับ Real-time communication
- **Database:** Amazon DynamoDB จำนวน 3 ตาราง (Users, Decks, RoomStateTable)
- **Security & Roles:** ควบคุมสิทธิ์การรันทั้งหมดผ่าน `LabRole`

4. โครงสร้างเว็บ (Frontend)

- **Components:** เน้นที่ `LorcanaBoard.tsx` (1,100+ บรรทัด) จัดการ State การเล่นเกม Local และทำหน้าที่เรนเดอร์บอร์ด
- **Services:** `src/services/websocket.ts` จัดการการเชื่อมต่อไปยัง AWS WebSocket API พร้อมส่ง Event การขยับการ์ด
- **Data Flow:** เมื่อผู้เล่นขยับการ์ด (Drag/Drop) คอมโพเนนต์จะอัปเดต UI ทันที (Optimistic Update) และเรียก Service เพื่อส่ง Payload `CARD_MOVED` เข้าสู่ AWS WebSocket API ซึ่งจะส่งต่อ (Relay) ไปยังผู้เล่นอื่นในห้องเดียวกันแบบ Real-time

5. สิ่งที่ต้องทำต่อ (Next Steps สำหรับ Sprint 4-5)

1. **Deck Analyzer (Sprint 4):** พัฒนาระบบวิเคราะห์ความสมดุลแบบ Asynchronous (ใช้ AWS SQS + Lambda)
2. **Frontend Integration:** นำหน้า Deck Builder มาต่อเข้ากับ AWS API Gateway จริง
3. **Refinement & Testing:** แก้ไขข้อบกพร่องเล็กน้อย ปรับปรุงประสิทธิภาพการเชื่อมต่อ WebSocket
4. **จัดทำรูปเล่ม Stage 2:** รวบรวม Diagram (Architecture) และผลทดสอบ เพื่อส่งมอบงานตามกำหนดการ

6. ผลการ Audit & Fix (14 ส.ค. 2569)

การตรวจสอบความถูกต้องของ Sprint 1-3 ด้วยการทดสอบจริงบน AWS Learner Lab พบและแก้ไข 2 ปัญหา:

ปัญหา	สาเหตุ	การแก้ไข	ผลทดสอบหลังแก้
GET /decks และ DELETE /decks/{id} ตอบ 405	API Gateway integration payload-format-version 2.0 แต่ Lambda เขียนด้วยรูปแบบ event v1 (event.httpMethod)	ใช้ เปลี่ยน integration เป็น 1.0 + แต่ redeploy stage	GET /decks → 200
POST /decks ถูกส่งไปยัง Lambda lorcana-auth-register	Integration mapping สลับ (integration ถูกสร้างด้วยลำดับไม่ตรงกับ route)	แก้ IntegrationUri ให้ชี้ lorcana-deck + redeploy	POST /decks → 201 บันทึกลง DynamoDB ได้จริง

ผลการทดสอบ End-to-End (ผ่าน URL จริง):

- POST /auth/register → 201 User registered successfully
- POST /auth/login → 200 Login successful + JWT token
- POST /decks → 201 บันทึกได้ลง DynamoDB
- GET /decks → 200 คืนรายการได้ที่บันทึก
- DELETE /decks/{id} → 200 ลบสำเร็จ
- WebSocket connect → 200 / JOIN_ROOM → 200 / CARD_MOVED → 200 (relay)

บทเรียน: เมื่อสร้าง API Gateway HTTP API ด้วย AWS_PROXY ต้องระบุ payload-format-version ให้ตรงกับ event format ที่ Lambda คาดหวัง และควรตรวจ integration mapping หลังสร้าง routes ทุกครั้ง

7. หมายเหตุ Security (รอบปรับปรุง Sprint 4)

- JWT_SECRET มีค่า fallback ฮาร์ดโค้ดในโค้ด — ควรย้ายเป็น AWS Secrets Manager หรือ environment variable ที่เข้มงวดกว่านี้ก่อนส่งงานจริง
- GET /decks รองรับ anonymous_guest (ไม่บังคับ auth) — เหมาะกับโหมด sandbox แต่ควรมี option บังคับ JWT สำหรับ production